Whose gate is it?

Every control you own lives in one of three contexts. The charter says what this project is about, the harness says how a turn happens, the factory says how work ships. Each holds different levers, changes at a different rate, and answers to a different team. The two boundaries agree almost everywhere, and the one place they disagree is where you spend all your arguments.

CHARTER · THE PROJECT DECLARES HARNESS · THE TURN CONTROLS FACTORY · DELIVERY SETTLES

WHO'S TALKING

Ian Johnson

- Staff engineer at **Parento**. Agents against a production codebase, daily.
- Founder of **Fulcorum** · fulcorum.com, video courses on shipping AI-generated code.
- Author of **Harness Engineering**, and of **ghola**, where these three contexts ship as a worker.

> I published this model, then corrected it twice. Both corrections are in this talk, at the slides where they cost me something.

THE NEXT THIRTY MINUTES

A map, a ladder, and an argument about who holds it.

ONE · THE MAP

Three contexts, the levers in each, and the one-way arrow between them that most designs point backwards.

TWO · THE RUNGS

Six places a gate can physically sit, plus two that sit off the ladder. One rule, shown at three of them.

THREE · THE OWNERS

Where the ownership line follows the context line, where it does not, and the rule that makes the overlap survivable.

ONE · THE ARGUMENT YOU KEEP HAVING

Everybody already owns all three. Almost nobody has drawn the lines.

- Platform ships a harness, and one repository's opinion keeps landing inside it where every other repository inherits it.
- Development writes a charter, and a third of it is trying to configure a pipeline it cannot reach.
- Both sides are right about the problem. Neither has agreed on the address, so the fix lands wherever the person who noticed it has commit access.

> The argument is never about the rule, only about which of you gets to change it next quarter.

THE MAP

Three contexts, and each one answers a different question.

CHARTER · THIS PROJECT

What is this repository about?

Claims about one codebase, written by the people inside it. Changes **constantly**.

HARNESS · DEVELOPMENT

How does a turn happen?

The shape of the work the agent does, everywhere it runs. Changes **rarely**.

FACTORY · DELIVERY

How does work ship?

Ordering, gates and the record, shared across every repository. Changes **very rarely**.

THE ARROW

The dependency points one way, and the arrow is the whole design.



> The factory knows about the harness. The harness must not know about the factory, because a harness that cannot run without a queue is a harness no developer can run at all.

THE LEVERS, NAMED

A component in the wrong context is the common defect.

CHARTER

`rule · hook · skill`
`command · agent`
`predicate · convention`

changes constantly

HARNESS

`prompt · tool · policy`
`budget · context`
`phase`

changes rarely

FACTORY

`stage · gate · guard`
`ordering · record`

changes very rarely

An **eval** belongs to none of them and can sit in all three, because it measures what no rule can decide and it changes whenever the thing it measures does. Give it the lifecycle of its subject, never the lifecycle of its context.

THE CHEAPEST DIAGNOSTIC YOU OWN

Proposals should thin out with distance from the code.

- Most of what goes wrong is something one repository wanted and never wrote down, so a healthy quarter is mostly charter changes.
- A quarter with three factory changes and no charter ones tells you the distribution is wrong. It does not tell you the factory needs work.
- Those rates are a diagnostic and never a schedule. Nobody should be hitting a quota on harness edits.

> Count where last quarter's changes landed. That histogram is a map of who currently believes they own what.

TWO · WHERE A GATE CAN SIT

Six rungs, and the context changes under you three times.

0	CHARTER	**the rule.** Prose the model reads and applies. Free to write, free to ignore, and it enforces nothing.
2	CHARTER	**the hook.** The repository's own script, on an event. Cheap and mechanical, and the agent it governs can delete it.
3i	HARNESS	**the judging phase.** A phase inside the turn that reads and refuses. Holds rules no predicate can express.
3d	HARNESS	**the predicate.** Code in front of every call the model makes. Exact, cheap per call, blind to what it cannot parse.
4i	FACTORY	**the review stage.** A pass that reads the finished work. Sees everything the turn produced, and nothing about how.
4d	FACTORY	**the stage gate.** A gate on the diff and on what gets published. The last mechanical rung the factory controls.

WHY EVERY CONTEXT HOLDS TWO

There are two ways to settle a question, and each context can do both.

INFERENTIAL · A MODEL JUDGES

A model reads the work and returns a verdict. It decides what no predicate can express, it costs tokens, and it can be wrong in two directions: by missing a violation, and by inventing one.

DETERMINISTIC · CODE SETTLES

Code reads the work and settles it. Exact, cheap at every call, and blind to everything it cannot parse. It never surprises you, which is both halves of the trade.

> Making a verdict more exact inside one context costs a function. Moving it to the next context costs a boundary and a new blind spot.
> Take the cheap move first.

THE TWO THAT SIT OFF THE LADDER

Two rungs are not on the chain, and both stay off it for a reason.

1	TOOL GRANT	**It withholds a capability** rather than deciding a case, so a rule cannot move here and still be the same rule. Withhold the tool and there is nothing left to enforce against.
5	CI	**It sits outside** the process that produced the work, and outside the process that checked it. That independence is the entire product.

> Move your rules into CI and you have spent the one rung that could have told you the other five were lying.

ONE RULE, THREE ADDRESSES, THREE OWNERS

The same sentence, written three times by two teams.

```
# rung 0 · charter · development writes it, development deletes it
"Every outbound HTTP call goes through lib/http. No direct requests."

# rung 3d · harness · platform ships the predicate, development registers the rule
deny(Edit) if adds_import("requests") and path not under "lib/http/"

# rung 4d · factory · platform owns it, and nothing in the turn can reach it
stage_gate: grep -rn "^import requests" --include=*.py | reject_outside lib/http
```

One sentence, and the owner changes twice on the way down. Nobody writes all three, and none of the three files mentions that the other two exist.

BEFORE YOU PROMOTE ANYTHING

One predicate at two boundaries is not one rule enforced twice.

- An in-harness check sees **tool calls**. It watches the work being made, and it can refuse while the agent can still act on the refusal.
- A stage gate sees the **finished tree**. It watches what came out, including everything that arrived by a path the harness never saw.
- Move a rule up and delete what it used to sit on, and you have silently removed the rung that caught what the new one cannot see.

> Ask what each rung can see. Two rungs that see the same thing are redundancy. Two that see different things are coverage.

THREE · WHOSE LEVER IS IT

Two lines, drawn by two different questions.

THE CONTEXT LINE

What does this thing govern? One project, one turn, or one delivery. I have written before that this line has nothing to do with ownership, and that is still true.

THE OWNERSHIP LINE

Who can change it, and who gets paged when it is wrong? A second line, laid over the first, and answered by a different fact about your org.

> They agree at the charter and they agree at the factory. They cross in the harness, which is why the harness is where every org-design argument actually lives.

WHERE THE LINES AGREE · ONE

The charter belongs to the people who own the code it describes.

- Every line is a claim about one repository, and only the team inside it knows whether the claim is still true this week.
- It changes constantly, so routing those changes through another team's queue prices the charter out of being maintained.
- Platform still owns two things here: the format the charter has to parse as, and the budget it has to fit in. Never the sentences.

> A charter written by a team that does not read the codebase is a list of guesses, and guesses are what a charter is supposed to replace.

WHERE THE LINES AGREE · TWO

The factory belongs to whoever gets paged when delivery breaks.

- Stages, gates, guards, ordering and the record are one pipeline shared by every repository, so every change is a change for all of them at once.
- A gate has to sit out of reach of the work it judges. In practice that means out of reach of the repository as well, not only out of reach of the agent.
- This is the context where "who can edit it" stops being an org-chart question and becomes a security property.

> A gate the repository can edit is a gate the repository has already passed.

WHERE THEY CROSS

Inside the harness, the split runs through the component list.

COMPONENT	PLATFORM OWNS	DEVELOPMENT OWNS
`prompt`	the skeleton and the phase contract	what this repository contributes to it
`tool`	which tools exist, and what they do	which of them a phase gets here
`policy`	the floor every repository inherits	tightening it, and only tightening it
`budget`	the ceiling, because it is money	nothing
`context`	the loading mechanism and its limits	what this repository loads into a turn
`phase`	the shape of a turn	nothing

THE RULE THAT MAKES THE OVERLAP SURVIVABLE

Platform sets the floor. Development tightens and never loosens.

- One direction is free. A repository asking for a stricter rule than the fleet is a repository doing your job for you, so let it, and log that it did.
- The other direction is a bypass wearing a config file. A repository that can loosen the floor has no floor, only a default.
- Every loosening that is genuinely legitimate gets a named escape hatch that the harness counts. A spike in that counter is the floor asking to be changed.

> Too loose is a blast radius. Too tight is a bypass flag by Thursday. A floor you can only tighten is how you get neither.

TACODA.DEV/NARROW-DOOR

WHERE SECURITY AND COMPLIANCE LAND

Org standards split by what they govern, not by who wrote them.

- A standard about how code gets **written** belongs in the harness, because it has to reach every turn, including the one a developer runs locally at 11pm.
- A standard about how code **ships** belongs in the factory, at the delivery rungs, where it can see the finished tree.
- Both are org-layer rules with the same author. They live in different contexts because they govern different things.

> Security hands you one document. Half is harness policy, half is a stage gate, and shipping it as one file puts one half in the wrong place.

GETTING IT BACKWARDS

Four ways this breaks, and each one has a tell you can look up today.

PLATFORM WRITES THE CHARTER

Rules that describe no repository in particular. **The tell:** a charter nobody has edited in a quarter, in a codebase that ships weekly.

DEVELOPMENT OWNS THE STAGE GATE

The gate sits inside its own blast radius. **The tell:** a gate with a perfect record, which usually means it has never been allowed to refuse.

A REPO RULE LANDS IN THE HARNESS

Every other repository inherits an opinion it never asked for. **The tell:** a policy with one team's service name written inside it.

THE HARNESS READS THE FACTORY

The harness stops running on a laptop. **The tell:** a developer cannot answer what their own rules do to a single turn.

THE TEST, FOR ANYTHING YOU ADD

Three questions, and the second one is not the first one.

1 · WHAT DOES IT GOVERN

This project, a turn, or a delivery. That answer picks the context, and it is a fact about the rule rather than about your team.

2 · WHO GETS PAGED

That answer picks the owner. Ask it separately, out loud, because the two answers come apart in the harness and nowhere else.

3 · CAN THE GOVERNED EDIT IT

Not whether they would. Whether they can. If they can, you own a convention, and you should stop calling it a gate.

MONDAY

One move each, and they are deliberately different moves.

IF YOU RUN THE PLATFORM

- Publish the floor as one versioned file, then count who tightens it and who asks you to loosen it.
- Switch the factory off and run one turn. If the harness cannot start, your arrow points the wrong way.

IF YOU WRITE THE CODE

- Find the lines in your charter that are trying to configure a pipeline, and move them or delete them.
- For your loudest rule, name the rung it sits on. If the answer is prose and you expected a gate, that gap is your next week.

BEFORE YOU TAKE ANY OF THIS HOME

What I have not measured, said plainly.

- The change rates come from my own repositories and from one production codebase. I have not counted them across teams, so treat them as a shape and not as a benchmark.
- The harness split on the earlier slide is where I have changed my mind most, and that table is its third version.
- The reviewing stage sat in my model as a capability for a long time before I noticed it was also a rung. Everything downstream of that mistake was wrong for about a year.

> If a model like this has never cost its author a correction, you are looking at a diagram rather than at something anyone ran.

A control has a context and an owner, and those are two different answers.

Pick one rule your team argues about. Name the context it governs, then name who gets paged when it is wrong, and write both down where the other team can read them. If those two answers land on the same group, the rule is already home. If they do not, you are in the harness, and now you know what the argument was about.

CONTACT

Find me, take the model.

THE MODEL

tacoda.dev/constraint-engineering

contexts, families, both ladders

THE RUNGS

tacoda.dev/layers-and-promotion

one rule at six rungs, measured

THE FLOOR

tacoda.dev/narrow-door

permissions, and where they bite

FULCORUM

fulcorum.com

EMAIL

ian@fulcorum.com

SITE · LINKEDIN · GITHUB

tacoda.dev
linkedin.com/in/tacoda
github.com/tacoda